



.Net

Teoría 12

Web Apis con Arquitectura Limpia



Descargar la solución EscuelaApi proporcionada por la cátedra



1. Abrir el Visual Studio Code en la carpeta de la solución.
2. Inspeccionar los proyectos que la conforman, las carpetas y archivos de código para entender cómo se articulan todos los componentes.

The screenshot shows the Visual Studio Code interface with the EscuelaApi solution open. The Explorer pane on the left displays the project structure:

- ESCUELA-API
 - EscuelaApi.Aplicacion
 - Abstracciones
 - bin
 - CasosDeUso
 - Dtos
 - Excepciones
 - obj
 - EscuelaApi.Aplicacion.csproj
 - EscuelaApi.Dominio
 - Abstracciones
 - bin
 - Entidades
 - Alumno.cs (selected)
 - Examen.cs
 - Usuario.cs
 - Excepciones
 - obj
 - ValueObjects
 - EscuelaApi.Dominio.csproj
 - EscuelaApi.Infraestructura
 - bin
 - OUTLINE
 - C# PROJECT DETAILS
 - TIMELINE

The main editor shows the code for `Alumno.cs` in the `EscuelaApi.Dominio > Entidades > Alumno.cs` namespace. The code defines the `Alumno` class, which inherits from `Entidad`. It includes properties for `Nombre` and `Email`, and a constructor that validates the `Nombre` property. The code is as follows:

```
1 namespace EscuelaApi.Dominio;
2
3 public class Alumno: Entidad
4 {
5     public string Nombre { get; private set; } = "";
6
7     public DireccionEmail? Email { get; private set; }
8
9     public Alumno(string nombre, DireccionEmail? email = null)
10    {
11        if (string.IsNullOrEmpty(nombre))
12        {
13            throw new DominioException("El nombre no puede ser nulo ni estar vacío");
14        }
15
16        Nombre = nombre;
17        Email = email;
18    }
19
20    protected Alumno() { } // Constructor vacío (lo utilizará Entity Framework)
21
22    // Otros métodos de la entidad Alumno que implementan su comportamiento
23    // permitiendo cambiar su estado y mantener su propia consistencia
24    // es decir, las invariantes de la entidad (reglas que siempre deben ser
25 }
```

Resumen de cuestiones novedosas que encontramos en la solución EscuelaApi

Antes de exponer nuestro sistema a la web mediante una **API**, es necesario comprender algunos aspectos que introdujimos en nuestra arquitectura respecto a implementaciones anteriores.

Estas decisiones de diseño nos permiten tener un código más limpio, seguro y alineado con el **Diseño Guiado por el Dominio (DDD)**.

Patrón Unit of Work

Patrón Unit of Work + ciclo de vida Scoped
=
Transacciones atómicas en la web

En aplicaciones complejas, un **Caso de Uso** a menudo necesita interactuar con múltiples **repositorios**. Todas esas operaciones deben guardarse en la base de datos como una **única transacción atómica** (todo o nada). **El patrón Unit of Work** nos da esa garantía.

Patrón Unit of Work

Inversión de dependencias en acción:

La capa de aplicación solo conoce la interfaz

`IUnidadDeTrabajo` con un único método: `GuardarCambios()`.

```
// EscuelaApi.Aplicacion/Abstracciones/IUnidadDeTrabajo.cs
public interface IUnidadDeTrabajo
{
    void GuardarCambios();
}
```

Patrón Unit of Work

El secreto de la infraestructura:

Los repositorios y la implementación concreta de `UnidadDeTrabajo` reciben exactamente la **misma instancia** de `EscuelaContext`. Los repositorios acumulan cambios en la memoria de ese contexto, y la `Unidad de Trabajo` da la orden final de guardar.

```
// EscuelaApi.Infraestructura/UnidadDeTrabajo/UnidadDeTrabajo.cs
public class UnidadDeTrabajo : IUnidadDeTrabajo
{
    private readonly EscuelaContext _context;
    public UnidadDeTrabajo(EscuelaContext context)
    {
        _context = context ?? throw new ArgumentNullException(nameof(context));
    }
    public void GuardarCambios()
    {
        _context.SaveChanges();
    }
}
```

Patrón Unit of Work

Ciclo de vida Scoped (clave en la web):

- En el contenedor DI de `Program.cs` registramos estos servicios como **Scoped**. Así, por cada petición HTTP se crea un único `EscuelaContext`, evitando mezclar datos entre usuarios.

```
// EscuelaApi.WebApi/Program.cs (fragmento)
builder.Services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();
builder.Services.AddScoped<IAlumnoRepository, AlumnoRepository>();
builder.Services.AddScoped<IExamenRepository, ExamenRepository>();
```


Un caso de uso usando Unit of Work Orquestación en la Capa de Aplicación

Ejemplo real: EliminarAlumnoUseCase

- Este caso de uso necesita eliminar un alumno y **todos sus exámenes asociados** en una sola transacción.

```
// EscuelaApi.Aplicacion/CasosDeUso/EliminarAlumnoUseCase.cs (fragmento)
public EliminarAlumnoResponse Ejecutar(EliminarAlumnoRequest request, Guid idUsuario)
{
    // 1. Validar autorización, existencia, etc.
    // 2. Obtener los exámenes del alumno
    var examenes = examenRepo.ObtenerPorAlumno(request.AlumnoId);
    // 3. Orquestación: Eliminar cada examen
    foreach (var examen in examenes)
        examenRepo.Eliminar(examen.Id);
    // 4. Orquestación: Eliminar al alumno
    alumnoRepo.Eliminar(request.AlumnoId);
    // 5. ¡TODO se guarda (o no) en una única transacción!
    uow.GuardarCambios(); // ← Punto clave
    return new EliminarAlumnoResponse(request.AlumnoId);
}
```

Un caso de uso usando Unit of Work Orquestación en la Capa de Aplicación

Observaciones didácticas:

- El caso de uso **no sabe** que la base de datos es **SQLite**, ni que hay transacciones. Solo pide **GuardarCambios()**.
- El patrón **Unit of Work** evita que un error a mitad de camino deje la base de datos en un estado intermedio. O se guarda todo, o no se guarda nada. Permite que múltiples repositorios trabajen en armonía.

El fin de los métodos Factory para hidratar entidades

Problema anterior (repositorios en archivos TXT):

- Solíamos necesitar un método estático `Reconstruir()` para instanciar entidades desde una fuente de datos externa, evitando que el constructor público volviera a generar un nuevo `Guid` o ejecutara validaciones que ya no correspondían.

Solución con EF Core:

- EF Core utiliza **reflexión** para "hidratar" los objetos. Solo necesita un **constructor sin parámetros con visibilidad `protected` o `private`** (`protected Alumno() { }`). El constructor público queda reservado exclusivamente para el *nacimiento* de nuevas entidades.

La eliminación del método "Update" en los repositorios y la mutación del estado

- Actualizar datos **no es responsabilidad de la infraestructura** (el repositorio). El repositorio es solo una colección; no debe tomar decisiones. La responsabilidad recae en la **Capa de Aplicación** (el Caso de Uso con su Unit of Work) que orquesta el cambio.

Ejemplo real: ModificarNotaDeExamenUseCase

```
// EscuelaApi.Aplicacion/CasosDeUso/ModificarNotaDeExamenUseCase.cs
public ModificarNotaResponse Ejecutar(ModificarNotaRequest request, Guid idUsuario)
{
    // 1. Obtener la entidad "viva" desde el repositorio
    var examen = examenRepo.ObtenerPorId(request.ExamenId);
    if (examen == null) throw new EntidadNoEncontradaException(...);

    // 2. Ejecutar el comportamiento del dominio (se validan reglas internas)
    examen.ModificarNota(request.NuevaNota); // ← muta el estado

    // 3. Actualizar cambios: EF Core genera automáticamente un UPDATE solo de la columna 'Nota'
    uow.GuardarCambios();
    return new ModificarNotaResponse(examen.Id);
}
```



La eliminación del método "Update" y la mutación del estado

¿Qué pasó detrás de escena?

- EF Core mantiene un **Change Tracker** que vigila la entidad examen.
- Cuando llamamos a **ModificarNota**, el **tracker** detecta que la propiedad **Nota** cambió.
- Al hacer **SaveChanges()**, genera una sentencia **SQL UPDATE Exámenes SET Nota = ... WHERE Id = ... solo para esa columna.**

Beneficio didáctico:

Al no tener un método **Update** en el repositorio, **forzamos** a los desarrolladores a seguir el flujo correcto del **DDD**:

```
repositorio.Obtener → entidad.MutarEstado → uow.GuardarCambios
```

Evolución del DbContext y el Composition Root

Cambio clave: Hemos eliminado el método `OnConfiguring` de `EscuelaContext`. El contexto **ya no sabe qué motor ni qué cadena de conexión usar**. Todo se define en el `Composition Root` (`Program.cs`)

Nuevo constructor (solo recibe opciones):

```
// EscuelaApi.Infraestructura/Datos/EscuelaContext.cs
public class EscuelaContext : DbContext
{
    // Las opciones (motor, cadena de conexión) vienen de fuera
    public EscuelaContext(DbContextOptions<EscuelaContext> options) : base(options) { }

    public DbSet<Alumno> Alumnos { get; set; }
    public DbSet<Examen> Exámenes { get; set; }
    public DbSet<Usuario> Usuarios { get; set; }

    // OnConfiguring ya no existe aquí
}
```



Repositorios Genéricos y `_context.Set<T>()`

- **Problema:** Queremos una clase base `Repository<T>` que sirva para `Alumno`, `Examen` y cualquier otra entidad. No podemos escribir `_context.Alumnos.Add(...)` porque no sabemos si `T` es `Alumno`, `Examen` o `Usuario`.
- **Solución:** El método `Set<T>()` de `DbContext` inspecciona dinámicamente el modelo y devuelve el `DbSet` correspondiente al tipo `T`.

```
// EscuelaApi.Infraestructura/Repositorios/Repository.cs (fragmento)
public class Repository<T> : IRepository<T> where T : Entidad
{
    . . .
    public void Agregar(T entidad)
    {
        _context.Set<T>().Add(entidad);    // ← Set<T>() resuelve el DbSet correcto
    }

    public T? ObtenerPorId(Guid id)
    {
        return _context.Set<T>().Find(id);
    }
}
```

Conceptos clave que hemos visto en esta implementación

- ✓ **Unit of Work + Scoped** → Transacciones atómicas en peticiones web.
- ✓ **Constructores protegidos** → Hidratación de entidades por **EF Core** sin perder reglas de negocio.
- ✓ **No más Update()** en los repositorios → Mutación del estado con **Change Tracker**.
- ✓ **DbContext sin OnConfiguring** → Configuración desde el **Composition Root**.
- ✓ **Repositorios genéricos con Set<T>()** → Reutilización segura

Web APIs y Minimal APIs en .NET

Exponiendo nuestra Arquitectura
Limpia al mundo

El nuevo punto de entrada: La Web API

- **El límite de la Consola:** Hasta ahora, nosotros éramos los únicos usuarios de nuestro sistema ejecutando proyectos de consola o pruebas.
- **El Mundo Real:** Nuestro Dominio y Casos de Uso necesitan ser consumidos por múltiples clientes (Apps Móviles, Frontend Web, otros sistemas).
- **La Solución:** Crear una capa de Presentación Web (API) que reciba peticiones por la red y las traduzca a llamadas a nuestra Capa de Aplicación.

El nuevo punto de entrada: La Web API

- **El límite de la Consola:** Hasta ahora, nosotros

Recordemos la **regla de oro** de la **Arquitectura Limpia**: la **interfaz de usuario** o el mecanismo de entrega **es un "detalle"**.

Vamos a cambiar nuestra consola por una interfaz que habla el idioma universal de internet, sin tocar ni una línea de nuestro Dominio.

traduzca a llamadas a nuestra **Capa de Aplicación**.



El Protocolo HTTP: El idioma de la Web

Comunicación basada en el modelo **Cliente-Servidor**.

- **Request (Petición):** El cliente pide algo indicando una **URL** y una intención.
- **Response (Respuesta):** El servidor devuelve un resultado, generalmente en formato **JSON**.
- **Los Verbos HTTP (La intención):**
 - **GET:** "Traer información" (Solo lectura).
 - **POST:** "Crear algo nuevo con estos datos".
 - **PUT / PATCH:** "Actualizar este recurso".
 - **DELETE:** "Eliminar este recurso".

Códigos de Estado HTTP (Status Codes)

Toda respuesta del servidor incluye un código numérico estandarizado que resume el resultado de la operación:

● 2xx (Éxito):

- **200 OK** (Salió bien).
- **201 Created** (Salió bien y se creó el recurso exitosamente).

● 4xx (Error del Cliente):

- **400 Bad Request** (Datos inválidos o regla de negocio violada).
- **401 Unauthorized** (Falta autenticación) / **403 Forbidden** (Faltan permisos).
- **404 Not Found** (El recurso solicitado no existe).

● 5xx (Error del Servidor):

- **500 Internal Server Error** (Excepción no controlada en nuestro código).

Minimal APIs: Nuestro Proyecto Anfitrión

- **ASP.NET Core** es el framework web oficial de Microsoft.
- **Minimal APIs:** Un enfoque moderno que permite crear endpoints **HTTP** con muy pocas líneas de código, sin configuraciones pesadas ni "**Controladores**" gigantes.
- En nuestra solución, esta capa está representada por el proyecto **EscuelaApi.WebApi**.
- Este proyecto actúa como el **Composition Root**: es el único lugar que conoce a todas las demás capas (**Dominio**, **Aplicación** e **Infraestructura**) para poder unirlos.

Minimal APIs: Nuestro Proyecto Anfitrión

- *ASP.NET Core* es el framework web oficial de Microsoft.

- *Minimal APIs* es un nuevo tipo de API que permite crear aplicaciones web más simples y fáciles de mantener.
- Elegimos *Minimal APIs* porque nos permite ver claramente la tubería (pipeline) *HTTP* sin el "ruido" visual que traen otros enfoques más antiguos de *ASP.NET*.
- El proyecto *Minimal APIs* se centra en la simplicidad y la claridad.

(Dominio, Aplicación e Infraestructura) para poder unirlos.



Inspeccionando el Composition Root (Program.cs del proyecto Web)

- El archivo principal ya tiene configurada la Inyección de Dependencias. ¡Aquí unimos las piezas!

```
var builder = WebApplication.CreateBuilder(args);

// A. Base de Datos (Leemos appsettings.json)
var connString = builder.Configuration.GetConnectionString("EscuelaDb");
builder.Services.AddDbContext<EscuelaContext>(opt => opt.UseSqlite(connString));

// B y C. Infraestructura y Seguridad (Scoped)
builder.Services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();
// ... repositorios y AutorizacionService ...

// D. Capa de Aplicación (Casos de Uso)
builder.Services.AddScoped<ConsultarExamenAlumnoUseCase>();
// ...
```


Externalizando la Configuración (appsettings.json)

- Ya no "*hardcodeamos*" contraseñas ni rutas de archivos en el código fuente. Todo va al archivo de configuración.
- En nuestro proyecto, el archivo `appsettings.json` define la conexión.

```
{  
  "ConnectionStrings": {  
    "EscuelaDb": "Data Source=Escuela.sqlite"  
  },  
  "Logging": { ... }  
}
```

- En Program.cs, el objeto builder lee este archivo automáticamente.
- Extraemos el valor usando el método `GetConnectionString`:

```
var connectionString = builder.Configuration.GetConnectionString("EscuelaDb");
```

La Magia de AddDbContext

- ¿Cómo llega esa cadena de conexión a nuestra capa de Infraestructura sin romper la Arquitectura Limpia?

```
builder.Services.AddDbContext<EscuelaContext>(opciones =>  
    opciones.UseSqlite(connectionString));
```

- **Ciclo de vida:** `AddDbContext` registra automáticamente nuestro contexto con un ciclo de vida **Scoped**.
- **Inyección de Opciones:** El delegado `opt => opt.UseSqlite(...)` empaqueta la orden de usar el motor `SQLite` junto con la ruta del archivo.
- **El Destino:** El `contenedor DI` inyectará este paquete completo en el constructor de `EscuelaContext`, eliminando por completo la necesidad del viejo método `OnConfiguring`.



Inicialización y Prueba de la API

- Antes de que el servidor empiece a escuchar peticiones, nos aseguramos de que la base de datos exista y dejamos un endpoint de prueba.

```
var app = builder.Build();

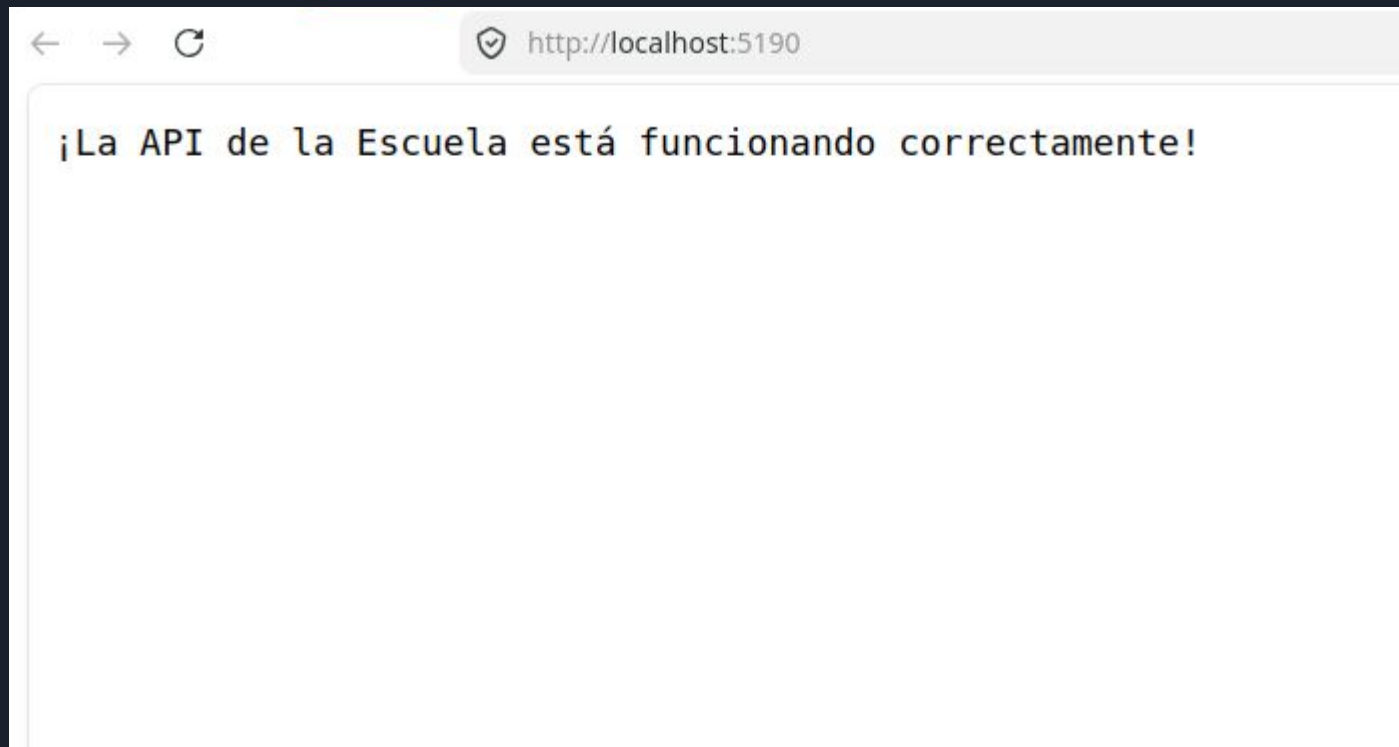
// Bloque 2: Simulamos un Scope temporal para inicializar SQLite
// porque EscuelaContext fue registrada como "Scoped"
using (var scope = app.Services.CreateScope()) {
    var context = scope.ServiceProvider.GetRequiredService<EscuelaContext>();
    EscuelaSqlite.Inicializar(context);
}

// Bloque 3: Endpoint de Prueba (Sanity Check)
app.MapGet("/", () => "¡La API de la Escuela está funcionando!");

app.Run(); // Arranca el servidor web (Kestrel)
```



Ejecutar la aplicación y verificar que la Web Api esté funcionando





Organizando las Rutas - Agregar en program.cs las siguientes líneas de código



En lugar de repetir `/api/alumnos` en cada endpoint, `Minimal APIs` nos permite agrupar rutas lógicamente usando `MapGroup`:

. . .

```
// Creamos un grupo base para todo lo relacionado a los alumnos  
var alumnosApi = app.MapGroup("/api/alumnos");
```

```
// A partir de aquí, las rutas se "enganchan" a ese prefijo.  
// Ejemplo: Esto responderá en GET /api/alumnos/  
// alumnosApi.MapGet("/", ...);
```

```
app.Run();
```



Nuestro primer Endpoint: Listar todos los Alumnos



Exponer el caso de uso `ConsultarAlumnosUseCase` para ver qué datos tenemos en la base. Como no requiere parámetros, la ruta será exactamente la base de nuestro grupo

. . .

```
// Ruta resultante: GET /api/alumnos/  
alumnosApi.MapGet("/", (ConsultarAlumnosUseCase useCase) =>  
{  
    // Instanciamos el DTO vacío  
    var request = new ConsultarAlumnosRequest();  
  
    var response = useCase.Ejecutar(request);  
    return Results.Ok(response);  
});
```

Injectado por el
Contenedor DI

```
app.Run();
```



Nuestro primer Endpoint: Listar todos los Alumnos



¿Qué hace exactamente `Results.Ok(response)`?

1. **Status Code:** Establece el código de respuesta HTTP en `200 OK` (Éxito).
2. **Serialización:** Toma nuestro objeto `DTO` de `C#` (`response`) y lo traduce automáticamente a texto en formato `JSON`.
3. **Cabeceras:** Le avisa al navegador (mediante el header `Content-Type: application/json`) el formato en el que viajan los datos para que sepa cómo interpretarlos.

```
return Results.Ok(response);  
});
```

```
app.Run();
```



Nuestro primer Endpoint: Listar todos los Alumnos



```
http://localhost:5190/api/alumnos

JSON  Datos en bruto  Encabezados

Guardar  Copiar  Contraer todo  Expandir todo  Filtrar JSON

▼ alumnos:

  ▼ 0:
    id:      "0ec0b598-f928-4c79-9e5f-bc31c23fd1bd"
    nombre:  "Laura"
    email:   "laura@hotmail.com"

  ▼ 1:
    id:      "4de8c950-dd41-4046-9cd2-c73dbb2061b5"
    nombre:  "Ana"
    email:   "ana@gmail.com"

  ▼ 2:
    id:      "f68d46b3-d068-4fbe-8d0e-0e651fb9046e"
    nombre:  "Juan"
    email:   "juan@gmail.com"
```




Endpoints con Parámetros en la URL

Agregar en program.cs las siguientes líneas de código



Queremos ver los exámenes de un alumno en particular. El framework puede extraer el **ID** directamente desde la **URL** y pasárselo a nuestro código:

...

Ejecutar y probar
en el navegador

```
// Ruta resultante: GET /api/alumnos/{alumnoId}/exámenes
alumnosApi.MapGet("/{alumnoId:guid}/exámenes", (
    Guid alumnoId,                                // 1. Extraído de la URL
    ConsultarExámenesAlumnoUseCase useCase) =>    // 2. Inyectado por el DI
{
    var request = new ConsultarExámenesAlumnoRequest(alumnoId);
    var response = useCase.Ejecutar(request);

    return Results.Ok(response);
});
```

```
app.Run();
```

Web APIs y Minimal APIs en .NET 10

← → ↻ http://localhost:5190/api/alumnos

JSON Datos en bruto Encabezados

Guardar Copiar Contraer todo Expandir todo Filtrar JSON

▼ alumnos:

- ▼ 0:
 - id: "0ec0b598-f928-4c79-9e5f-bc31c23fd1bd"
 - nombre: "Laura"
 - email: "laura@hotmail.com"
- ▼ 1:
 - id: "4de8c950-dd41-4046-9cd2-c73dbb2061b5"
 - nombre: "Ana"
 - email: "ana@gmail.com"
- ▼ 2:
 - id: "f68d46b3-d068-4fbe-8d0e-0e651fb9046e"
 - nombre: "Juan"
 - email: "juan@gmail.com"

f68d46b3-d068-4fbe-8d0e-0e651fb9046e

← → ↻ http://localhost:5190/api/alumnos/f68d46b3-d068-4fbe-8d0e-0e651fb9046e/examenes

JSON Datos en bruto Encabezados

Guardar Copiar Contraer todo Expandir todo Filtrar JSON

▼ examenes:

- ▼ 0:
 - id: "3c7e55c4-d3a2-424b-9ad4-2a649ad2fd20"
 - materia: "Álgebra"
 - nota: 10
 - fecha: "2021-05-24T00:00:00"
- ▼ 1:
 - id: "3e53d0d2-a320-4a07-a400-5fc23990ab47"
 - materia: "Inglés"
 - nota: 5
 - fecha: "2019-03-01T00:00:00"

Anatomía de una Ruta: Parámetros y Restricciones

Analicemos la ruta: `/api/alumnos/{alumnoid:guid}/exámenes`

Esta sintaxis utiliza **Route Constraints** (Restricciones de Ruta), dividida en dos partes por los dos puntos (:):

1. **El Nombre (alumnoid):** Es la variable. **Debe coincidir exactamente** con el nombre del parámetro en nuestra función Lambda para que el *Model Binding* sepa dónde inyectar el dato.
2. **La Restricción (guid):** Es el filtro de validación. Exige que el texto en esa posición sea un **Identificador Único Global** válido.
 - **Convención:** Siempre se escribe en **minúsculas** (:guid, :int, :bool), ya que son claves internas del motor de ruteo, no los tipos nativos de C#.

Anatomía de una Ruta: Parámetros y Restricciones

Analicemos la ruta: `/api/alumnos/{alumnoid:guid}/exámenes`

El "Escudo 404":

- Si un cliente solicita `/api/alumnos/pepito/exámenes`, el motor de ruteo detecta que "pepito" no es un `Guid`.
- **Resultado:** Rechaza la petición automáticamente devolviendo `HTTP 404 (Not Found)`, sin siquiera ejecutar nuestro código ni llegar al Caso de Uso.

El framework nos está ahorrando trabajo de validación. No tenemos que escribir un `if (!Guid.TryParse(...))` dentro de nuestro endpoint porque el servidor web actúa como un guardia de seguridad en la puerta, filtrando la "basura" antes de que llegue a nuestra aplicación.

OpenAPI, Scalar
y los verbos de escritura.

El límite del Navegador Web

- Hasta ahora, probamos nuestros endpoints (**GET**) escribiendo la URL en el navegador.
- **El problema:** Un navegador web convencional, por defecto, **solo puede realizar peticiones GET**.
- Si queremos enviar datos complejos al servidor para crear un alumno (**POST**), el navegador no nos sirve. Necesitamos construir un "Cuerpo" (**Body**) en la petición con formato **JSON**.
- **Soluciones tradicionales:** Usar herramientas de escritorio externas como **Postman**, **Insomnia** o ejecutar comandos **curl** en la terminal.
- **El problema de las herramientas externas:** Obligan a salir del entorno, exigen configuración manual y se desincronizan fácilmente si cambiamos el código.

El límite del Navegador Web

¿Cómo hacemos entonces para mandar un DTO con los datos para crear un alumno?

Vamos a integrar la herramienta de prueba **directamente dentro de nuestra API**, (accesible sólo en un entorno de desarrollo) simplificando el proceso y permitiendo probar funcionalidades con solo ejecutar el proyecto.

La solución estándar: OpenAPI

OpenAPI: Es una especificación estándar en la industria para describir interfaces de programación (APIs).

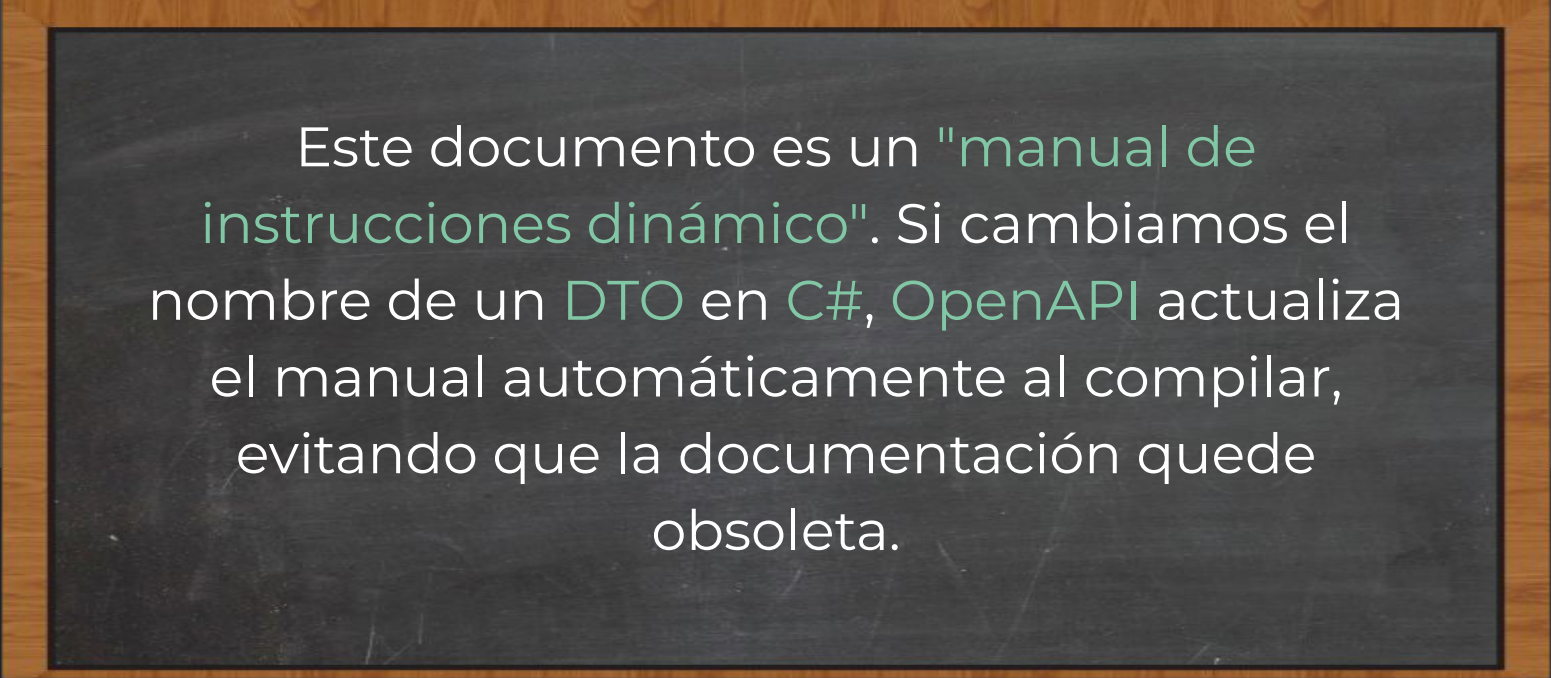
Funciona como un "contrato" o "mapa" dinámico:

- Lee nuestro código C#.
- Detecta qué rutas existen, qué verbos usan y qué parámetros necesitan.
- Genera un documento (un archivo JSON gigante) que describe toda nuestra API.

ASP.NET Core tiene soporte nativo para generar este documento automáticamente.

La solución estándar: OpenAPI

OpenAPI: Es una especificación estándar en la industria para describir interfaces de programación (APIs).



Este documento es un "manual de instrucciones dinámico". Si cambiamos el nombre de un DTO en C#, OpenAPI actualiza el manual automáticamente al compilar, evitando que la documentación quede obsoleta.

Nuestra interfaz interactiva

OpenAPI solo genera el "documento" (los datos crudos). Para que nosotros los humanos podamos leerlo y probarlo, necesitamos una interfaz gráfica (**UI**).

Históricamente en **.NET** se usaba *Swagger UI*.

El nuevo estándar: Hoy utilizaremos **Scalar**, una interfaz moderna, minimalista y mucho más potente que se integra a la perfección con **.NET 10**.

Nos permite:

- Ver todos los endpoints agrupados.
- Conocer exactamente el formato **JSON** que espera la **API**.
- Ejecutar las peticiones con un solo clic directamente desde el navegador.



Integrando OpenAPI y Scalar en el Proyecto



Agregar los paquetes correspondientes e inyectarlos en nuestro Program.cs.

1. En la terminal (dentro de la carpeta WebApi):

```
dotnet add package Microsoft.AspNetCore.OpenApi  
dotnet add package Scalar.AspNetCore
```



Agregar las líneas resaltadas en Program.cs



```
// *** Sección usings ***  
using Scalar.AspNetCore;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
// =====  
// BLOQUE 1: CONFIGURACIÓN DE SERVICIOS (El Contenedor DI)  
// =====
```

```
builder.Services.AddOpenApi();
```

```
// . . .
```

```
var app = builder.Build();
```

```
// Solo exponemos esto en modo Desarrollo (seguridad)  
if (app.Environment.IsDevelopment())  
{  
    app.MapOpenApi(); // Genera el archivo JSON interno  
    app.MapScalarApiReference(); // Levanta la interfaz gráfica en /scalar  
}
```

```
// . . .
```

```
app.Run();
```

Ejecutar y probar
en el navegador
<http://localhost:5190/scalar>

http://localhost:5190/scalar/

Search ^ K Ask AI

Introduction

EscuelaApi.WebApi

- / GET
- /api/alumnos GET
- /api/alumnos/{alumnoId}/exámenes GET

Endpoints de nuestra API

v1.0.0 OpenAPI 3.1.1

EscuelaApi.WebApi | v1

Download OpenAPI Document

Desde aquí se puede descargar la documentación de la API

Server

http://localhost:5190

Client Libraries

> Shell Ruby Node.js PHP Python More

C# HttpClient

EscuelaApi.WebApi

Operations

Generate MCP

Powered by Scalar

OpenAPI, Scalar y los verbos de escritura

The screenshot displays the Scalar API client interface in a web browser. The address bar shows the URL `http://localhost:5190/scalar/#tag/escuelaapiwebapi/GET/api/alumnos`. The left sidebar contains a search bar and a list of endpoints under the heading "EscuelaApi.WebApi". The endpoint `/api/alumnos` is selected and highlighted, with a red arrow pointing to it. The main panel shows the selected endpoint `GET /api/alumnos/{alumnoId}/exámenes`. A red box with the text "Probar el endpoint /api/alumnos" is overlaid on the main panel. Below the endpoint, the "Responses" section shows a `200 OK` response. A dark panel at the bottom contains a `curl` command: `curl http://localhost:5190/`. A red arrow points to the "Test Request" button in this panel. The bottom of the interface shows a "Generate MCP" button and the text "Powered by Scalar".

Search ^ K Ask AI

Introduction

EscuelaApi.WebApi

- / GET
- /api/alumnos GET**
- /api/alumnos/{alumnoId}/exámenes GET

GET /api/alumnos/{alumnoId}/exámenes

Probar el endpoint /api/alumnos

Responses

> 200 OK

GET /

1 `curl http://localhost:5190/`

Ask AI Agent Test Request

200

string

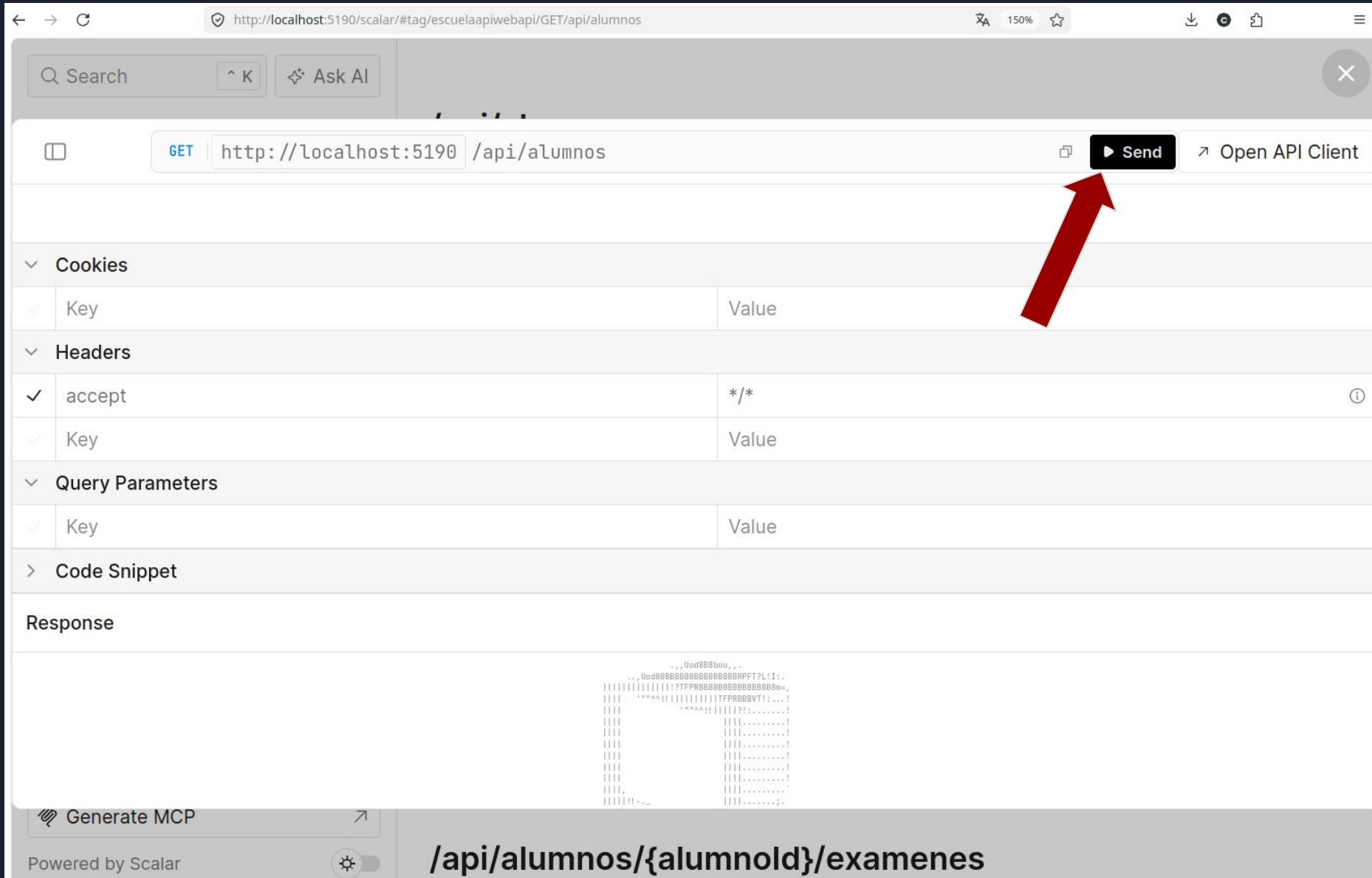
OK

Show Schema

Generate MCP

Powered by Scalar

OpenAPI, Scalar y los verbos de escritura



The screenshot shows the Scalar API client interface. At the top, the URL bar displays `http://localhost:5190/scalar/#tag/escuelaapiwebapi/GET/api/alumnos`. Below the URL bar, the request method is set to `GET` and the URL is `http://localhost:5190/api/alumnos`. A red arrow points to the `Send` button. To the right of the `Send` button is a link to `Open API Client`.

Below the request bar, there are sections for `Cookies`, `Headers`, `Query Parameters`, and `Code Snippet`. The `Headers` section is expanded, showing a table with the following data:

Key	Value
accept	*/*

The `Query Parameters` section is also expanded, showing a table with the following data:

Key	Value
-----	-------

At the bottom, there is a `Response` section. Below the `Response` section, there is a `Generate MCP` button. The footer of the interface shows `Powered by Scalar` and a URL `/api/alumnos/{alumnold}/examenenes`.

OpenAPI, Scalar y los verbos de escritura

GET http://localhost:5190/api/alumnos

224ms 200 OK

Open API Client

✓ Cookies

✓ Headers

✓ Query Parameters

✓ accept */*

✓ Key Value

✓ Key Value

✓ Key Value

✓ Cookies

✓ Request Headers 1

✓ Response Headers 4

✓ Body

application/json

Download

Preview Raw

```
1 {
2   "alumnos": [
3     {
4       "id": "0ec0b598-f928-4c79-9e5f-bc31c23fd1bd",
5       "nombre": "Laura",
6       "email": "laura@hotmail.com"
7     },
8     {
9       "id": "4de8c950-dd41-4046-9cd2-c73dbb2061b5",
10      "nombre": "Ana",
11      "email": "ana@gmail.com"
12    },
13    {
14      "id": "f68d46b3-d068-4fbc-8d0e-0e651fb9046e",
15      "nombre": "Juan",
16      "email": "juan@gmail.com"
17    }
18  ]
19 }
```

Copiar este id porque lo vamos a utilizar al testear el otro endpoint

> Code Snippet

OpenAPI, Scalar y los verbos de escritura

The image shows the Scalar API client interface. The browser address bar displays `http://localhost:5190/scalar/#tag/escuelaapiwebapi/GET/api/alumnos/{alumnoId}/exámenes`.

Endpoint 1: /api/alumnos

- Responses:** 200 OK
- Preview:** Shows a 200 status and "No Body".
- Test Request:** `curl http://localhost:5190/api/alumnos`

Endpoint 2: /api/alumnos/{alumnoId}/exámenes

- Path Parameters:** `alumnoId` string · uuid required
- Responses:** 200 OK
- Test Request:** `curl http://localhost:5190/api/alumnos/123e2...89b-12d3-a4...`

A red arrow points to the endpoint `/api/alumnos/{alumnoId}/exámenes` in the sidebar. Another red arrow points to the "Test Request" button in the preview for the second endpoint.

OpenAPI, Scalar y los verbos de escritura

The image shows the Scalar API client interface. The top bar displays the method **GET** and the URL `http://localhost:5190/api/alumnos/{alumnoId}/examenes`. A **Send** button is located in the top right corner, with a large red arrow pointing to it. Below the URL bar, the left sidebar contains sections for **Variables**, **Cookies**, **Headers**, and **Query Parameters**. The **Variables** section is expanded, showing a table with the following data:

Variable	Required	Value
alumnoId	Required	f68d46b3-d068-4fbc-8d0e-0e651fb9046e

A red arrow points from the **alumnoId** value to the **Send** button. Below the sidebar, a red box contains the text: "Escribir aquí el valor del parámetro **alumnoId**". The right sidebar shows the **Response** section, which is currently empty. At the bottom right, there is a **Send Request** button with a **CTRL** shortcut.

OpenAPI, Scalar y los verbos de escritura

GET http://localhost:5190 /api/alumnos/{alumnoId}/exámenes

162ms 200 OK

Variables

✓	alumnoid	Required	f68d46b3-d068-4fbe-8d0e-0e651fb9046e
---	----------	----------	--------------------------------------

Cookies

✓	Key	Value
---	-----	-------

Headers

✓	accept	*/*
✓	Key	Value

Query Parameters

✓	Key	Value
---	-----	-------

Body

application/json

```
1 {
2   "exámenes": [
3     {
4       "id": "3c7e55c4-d3a2-424b-9ad4-2a649ad2fd20",
5       "materia": "Álgebra",
6       "nota": 10,
7       "fecha": "2021-05-24T00:00:00"
8     },
9     {
10      "id": "3e53d0d2-a320-4a07-a400-5fc23990ab47",
11      "materia": "Inglés",
12      "nota": 5,
13      "fecha": "2019-03-01T00:00:00"
14    }
15  ]
16 }
```

Estos son los exámenes del alumno f68d46b3-d068-4fbe-8d0e-0e651fb9046e

Code Snippet



Agregar este endpoint para poder listar los usuarios



. . .

```
// Grupo para usuarios
```

```
var usuariosApi = app.MapGroup("/api/usuarios");
```

```
usuariosApi.MapGet("/", (ConsultarUsuariosUseCase useCase) =>  
{
```

```
    var request = new ConsultarUsuariosRequest();
```

```
    var response = useCase.Ejecutar(request);
```

```
    return Results.Ok(response);
```

```
});
```

. . .

Ejecutar y probar
en el navegador
<http://localhost:5190/scalar>

OpenAPI, Scalar y los verbos de escritura

GET http://localhost:5190/api/usuarios

205ms 200 OK

Request Headers

Key	Value
accept	*/*
Key	Value

Response Headers

application/json

Body

```
1 {
2   "usuarios": [
3     {
4       "id": "0048a7e3-be38-49db-a8d2-b38e206af863",
5       "nombre": "Admin del Sistema",
6       "email": "admin@escuela.edu.ar",
7       "rol": "Administrador"
8     },
9     {
10      "id": "a65761a2-0460-4ac8-af9b-cb2d7e89a024",
11      "nombre": "Roberto (Profesor)",
12      "email": "roberto@escuela.edu.ar",
13      "rol": "Profesor"
14    }
15  ]
16 }
```

Aquí está la lista de usuarios
Tomar nota del id del Administrador
porque lo necesitaremos en futuras pruebas

Code Snippet



Nuestro primer POST: Creando un Alumno

Agregar el using y la nueva ruta en Program.cs



```
using Microsoft.AspNetCore.Mvc; // Para [FromHeader]

. . .

// Ruta resultante: POST /api/alumnos/
alumnosApi.MapPost("/", (
    AgregarAlumnoRequest request, // 1. Extraído del Body (JSON)
    [FromHeader(Name = "X-Usuario-Id")] Guid idUsuario, // 2. Puente temporal para la seguridad
    AgregarAlumnoUseCase useCase) => // 3. Inyectado por DI
{
    var response = useCase.Ejecutar(request, idUsuario);

    // HTTP 201: Creado exitosamente
    return Results.Created($"{api/alumnos/{response.Id}", response);
});

. . .
```



Nuestro primer POST: Creando un Alumno



El *Results.Created*: a diferencia de *Results.Ok()*, cuando creamos algo, las buenas prácticas dictan devolver un **HTTP 201 Created** y la **URL** donde el cliente puede ir a buscar ese nuevo recurso.

del Body (JSON)
temporal para la seguridad
o por DI

```
// HTTP 201: Creado exitosamente  
return Results.Created($"api/alumnos/{response.Id}", response);  
});
```

...

Ejecutar y probar
en el navegador
<http://localhost:5190/scalar>

OpenAPI, Scalar y los verbos de escritura

Scalar API client interface showing the OpenAPI specification for a REST API.

Endpoint: /api/alumnos (POST)

Headers:

- X-Usuario-Id: string · uuid **required**

Request Body:

Body: RegistrarAlumnoRequest **required** application/json

Responses:

- 200 OK

Shell Curl:

```
POST /api/alumnos
1 curl http://localhost:5190/api/alumnos \
2 --request POST \
3 --header 'X-Usuario-Id: ' \
4 --header 'Content-Type: application/json' \
5 --data '{
6   "nombre": "",
7   "email": ""
8 }'
```

Test Request:

200

No Body

OK

Endpoint: /api/alumnos/{alumnoId}/exámenes (GET)

Path Parameters:

- alumnoId: string · uuid **required**

Shell Curl:

```
GET /api/alumnos/{alumnoId}/exámenes
1 curl http://localhost:5190/api/alumnos/123e4567-e89b-3
```


OpenAPI, Scalar y los verbos de escritura

POST http://localhost:5190 /api/alumnos

Send

Open API Client

Response

⌵ Cookies

Key	Value
-----	-------

⌵ Headers

✓ accept	*/*	ⓘ
✓ content-type	application/json	ⓘ
✓ X-Usuario-Id	Required 0048a7e3-be38-49db-a8d2-b38e206af8...	ⓘ
Key	Value	

⌵ Query Parameters

⌵ Request Body

JSON ⌵

```
1 {
2   "nombre": "",
3   "email": "",
4 }
```

Ingresar el Id de un usuario con el rol "Administrador"

Completar el nombre y el email del nuevo usuario, por ejemplo:

```
{
  "nombre": "María",
  "email": "maria@escuela.edu.ar"
}
```

Send Request CTRL ↵

OpenAPI, Scalar y los verbos de escritura

OpenAPI Client

POST http://localhost:5190 /api/alumnos Send

204ms 201 Created

⌵ Cookies

Key	Value
-----	-------

⌵ Headers

✓ accept	*/*	ⓘ
✓ content-type	application/json	ⓘ
✓ X-Usuario-Id	Required 0048a7e3-be38-49db-a8d2-b38e206af8...	ⓘ
Key	Value	

⌵ Query Parameters

⌵ Request Body

JSON ⌵

```
1 {  
2   "nombre": "María",  
3   "email": "maria@escuela.edu.ar"  
4 }
```

⌵ Cookies

⌵ Request Headers 3

⌵ Response Headers 5

⌵ Body Download

application/json Preview Raw

```
1 {  
2   "id": "645528e3-cf68-4c7f-a9d8-bf8e30aa2625"  
3 }
```

Id del nuevo alumno creado

⌵ Code Snippet

OpenAPI, Scalar y los verbos de escritura

GET http://localhost:5190 /api/alumnos Send Open API Client

23ms 200 OK

Cookies

Key	Value
-----	-------

Headers

Key	Value
accept	*/*
Key	Value

Query Parameters

Body Download Preview Raw

application/json

```
1 {
2   "alumnos": [
3     {
4       "id": "0ec0b598-f928-4c79-9e5f-bc31c23fd1bd",
5       "nombre": "Laura",
6       "email": "laura@hotmail.com"
7     },
8     {
9       "id": "4de8c950-dd41-4046-9cd2-c73dbb2061b5",
10      "nombre": "Ana",
11      "email": "ana@gmail.com"
12    },
13    {
14      "id": "f68d46b3-d068-4fbe-8d0e-0e651fb9046e",
15      "nombre": "Juan",
16      "email": "juan@gmail.com"
17    },
18    {
19      "id": "645528e3-cf68-4c7f-a9d8-bf8e30aa2625",
20      "nombre": "María",
21      "email": "maria@escuela.edu.ar"
22    }
23  ]
24 }
```

Code Snippet

Si listamos todos los alumnos vemos al nuevo alumno creado



Agregar la ruta para registrar un Examen (POST)



. . .

```
// Ruta resultante: POST /api/examenes/  
var examenesApi = app.MapGroup("/api/examenes");  
  
examenesApi.MapPost("/", (  
    RegistrarExamenRequest request,  
    [FromHeader(Name = "X-Usuario-Id")] Guid idUsuario,  
    RegistrarExamenUseCase useCase) =>  
{  
    var response = useCase.Ejecutar(request, idUsuario);  
  
    // HTTP 201: Creado exitosamente  
    return Results.Created($"api/examenes/{response.Id}", response);  
});
```

. . .

Ejecutar y probar
en el navegador
<http://localhost:5190/scalar>

Buscar el ID de un **alumno** existente y el ID del usuario **Profesor** para cargar una nota desde Scalar. Si intentan cargarlo con el ID del **Administrador**, el Caso de Uso lanzará nuestra **AutorizacionException**.



Agregar la ruta para modificar la nota de un Examen (PUT)



. . .

```
// Ruta resultante: PUT /api/examenes/{examenId}
examenesApi.MapPut("/{examenId:guid}", (
    Guid examenId, // Parámetro en la URL
    ModificarNotaRequest request, // El Body trae la NuevaNota
    [FromHeader(Name = "X-Usuario-Id")] Guid idUsuario,
    ModificarNotaDeExamenUseCase useCase) =>
{
    // Aseguramos que el ID de la URL y el del Body coincidan (buena práctica)
    if (examenId != request.ExamenId) return Results.BadRequest();

    var response = useCase.Ejecutar(request, idUsuario);

    // HTTP 200: OK
    return Results.Ok(response);
});
```

. . .



Agregar la ruta para modificar la nota de un Examen (PUT)

Ejecutar y probar
en el navegador
<http://localhost:5190/scalar>



```
...  
  
// Ruta resultante: PUT /api/examenes/{examenId}  
examenesApi.MapPut("/{examenId:guid}", (  
    Guid examenId, // Parámetro en la URL  
    ModificarNotaRequest request, // El Body trae la NuevaNota  
    [FromHeader(Name = "X-Usuario-Id")] Guid idUsuario,  
    ModificarNotaDeExamenUseCase useCase) =>  
{  
    // Aseguramos que el ID de la URL y el del Body coincidan (buena  
    if (examenId != request.ExamenId) return Results.BadRequest();  
  
    var response = useCase.Ejecutar(request, idUsuario);  
  
    // HTTP 200: OK  
    return Results.Ok(response);  
});  
  
...
```

Convención REST: El "Quién" vs. El "Qué"

- **La URL** siempre identifica al recurso a modificar (el "Quién").
- **El Body (JSON)** transporta el nuevo estado o los datos (el "Qué").

Nota: El ID en la URL permite que la infraestructura de red sepa exactamente qué recurso se está afectando sin necesidad de inspeccionar el contenido del mensaje.



Agregar la ruta para borrar un alumno (DELETE)



...

```
// Ruta resultante: DELETE /api/alumnos/{alumnoId}
alumnosApi.MapDelete("/{alumnoId:guid}", (
    Guid alumnoId,
    [FromHeader(Name = "X-Usuario-Id")] Guid idUsuario,
    EliminarAlumnoUseCase useCase) =>
{
    var request = new EliminarAlumnoRequest(alumnoId);
    var response = useCase.Ejecutar(request, idUsuario);

    // HTTP 200: OK (o podría ser 204 No Content)
    return Results.Ok(response);
});
```

...

La Unidad de Trabajo
(`UnitOfWork`) garantiza la
atomicidad: el Caso de Uso primero
elimina los exámenes asociados y
luego al alumno.

Ejecutar y probar
en el navegador
<http://localhost:5190/scalar>



El problema invisible... ¿Qué pasa si fallamos?



Tenemos todos los endpoints funcionando. Pero hagamos una prueba en **Scalar**:

- Intentar crear un alumno con el **email vacío**.
- Intentar modificar una nota y pongámosle un **15**.
- Intentar realizar una acción con un rol **incorrecto**.

¿Qué nos responde la API? ¡Un error **HTTP 500 (Internal Server Error)** cuando en realidad no es culpa del servidor sino del cliente, debería informarse **HTTP 400 Bad Request**.

- **El problema:** Para el servidor web, **DominioException** es un error fatal y el sistema "explota".
- **La solución:** Necesitamos un traductor global de excepciones.

POST

http://localhost:5190/api/alumnos

Send

Open API Client

71s 500 Internal Server Error

Cookies

Request Headers 3

Response Headers 4

Body

Key

Value

accept

/**

content-type

application/json

X-Usuario-Id

Required

0048a7e3-be38-49db-a8d2-b38e206af8...

Key

Value

Query Parameters

Key

Value

Request Body

JSON

1 {

2 "nombre": "José",

3 "email": ""

4 }

71s 500 Internal Server Error

Cookies

Request Headers 3

Response Headers 4

Body

1 EscuelaApi.Dominio.DominioException: El formato del email no es válido.

2 at EscuelaApi.Aplicacion.AgregarAlumnoUseCase.Ejecutar(AgregarAlumnoRequest

3 at Program.<c.<<Main>\$>b__0_4(AgregarAlumnoRequest request, Guid idUsuario

4 at lambda_method10(Closure, Object, HttpContext, Object)

5 at Microsoft.AspNetCore.Http.RequestDelegateFactory.<c__DisplayClass102_2.

6 --- End of stack trace from previous location ---

7 at Microsoft.AspNetCore.Diagnostics.DeveloperExceptionPageMiddlewareImpl.In'

8

9 HEADERS

10 =====

11 Accept: */*

12 Connection: keep-alive

13 Host: localhost:5190

14 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:151.0) Gecko/20100101 Firefox/1!

15 Accept-Encoding: gzip, deflate, br, zstd

16 Accept-Language: es

17 Content-Type: application/json

18 Cookie: .AspNetCore.Antiforgery.RuQmoXmVEtg=CfDJ8A_el4n3ek1Jn4j89mzEglnm8HbJ-9,

19 Origin: http://localhost:5190

20 Referer: http://localhost:5190/scaIar/

21 Content-Length: 38

22 x-usuario-id: 0048a7e3-be38-49db-a8d2-b38e206af863

23 DNT: 1

24 Sec-Fetch-Dest: empty

25 Sec-Fetch-Mode: cors

26 Sec-Fetch-Site: same-origin

Code Snippet

Error http 500

Agregando un alumno con email vacío

Middleware de excepciones global

El problema de las excepciones crudas

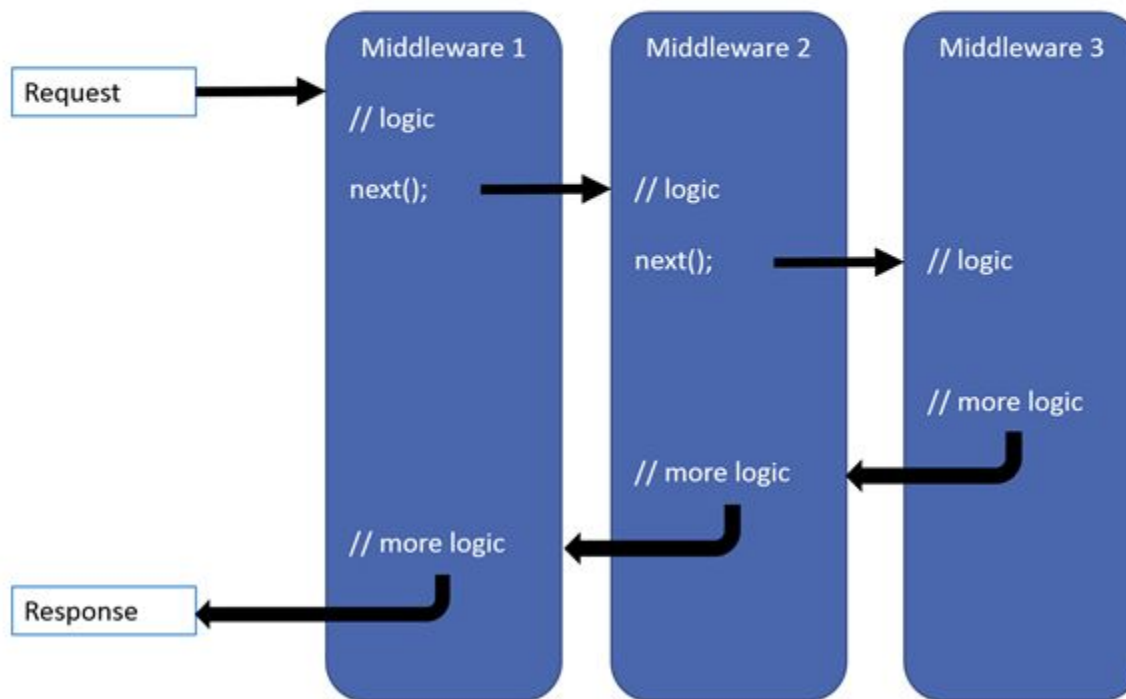
- Ya sabemos que nuestras entidades y casos de uso lanzan excepciones (ej: `EntidadNoEncontradaException`, `DominioException`) cuando se rompe una regla.
- **¿Qué hace el servidor web ante una excepción no atrapada?** Asume que el sistema falló catastróficamente, aborta la operación y devuelve un **HTTP 500 (Internal Server Error)**.
- **El error semántico:** Si se envía un `DTO` con una nota de "15", la culpa no es del servidor, ¡es del cliente! El código de respuesta debe ser **HTTP 400 (Bad Request)**.
- Llenar cada endpoint de bloques `try-catch` haría que el código sea repetitivo, sucio y difícil de mantener.

El Pipeline y los Middlewares

- ASP.NET Core funciona a través de un **Pipeline** (una tubería).
- Cada petición **HTTP** que ingresa viaja por esa tubería pasando a través de distintos filtros llamados **Middlewares** antes de llegar a nuestros endpoints.
- A la vuelta, la respuesta **HTTP** hace el camino inverso pasando de nuevo por los mismos filtros.
- **La estrategia:** Podemos colocar un "Middleware de Manejo de Excepciones" al principio del tubo. Si algún endpoint explota, la excepción viajará por el tubo hacia afuera y nuestro filtro la atrapará justo antes de que se envíe al cliente.

El Pipeline y los Middlewares

La canalización de solicitudes de ASP.NET Core consiste en una secuencia de delegados de solicitud a los que se llama de uno en uno. En el siguiente diagrama se muestra este concepto. El subproceso de ejecución sigue las flechas negras.





Crear nuestro "Traductor" (ExceptionHandler)



En .NET moderno, creamos una clase que implemente la interfaz `ExceptionHandler`. Crearemos esta clase en nuestro proyecto `EscuelaApi.WebApi`

```
using Microsoft.AspNetCore.Diagnostics;  
using Microsoft.AspNetCore.Mvc;  
using EscuelaApi.Dominio;  
using EscuelaApi.Aplicacion;
```

```
namespace EscuelaApi.WebApi;
```

```
public class ManejadorDeExcepcionesGlobales : ExceptionHandler  
{  
    public async ValueTask<bool> TryHandleAsync(  
        HttpContext httpContext, Exception exception, CancellationToken  
        cancellationToken)  
    {  
        // ... acá irá la lógica de traducción ...  
        return true;  
    }  
}
```

Retornar true al final del método es vital: le avisa a .NET "quedate tranquilo, yo ya me hice cargo de este problema, no tires el error 500 estándar".



Completar la lógica de traducción



```
var problemDetails = new ProblemDetails { Instance = httpContext.Request.Path };

if (exception is DominioException) {
    problemDetails.Title = "Error de validación del negocio";
    problemDetails.Status = StatusCodes.Status400BadRequest;
    problemDetails.Detail = exception.Message;
} else if (exception is EntidadNoEncontradaException) {
    problemDetails.Title = "Recurso no encontrado";
    problemDetails.Status = StatusCodes.Status404NotFound;
    problemDetails.Detail = exception.Message;
} else if (exception is AutorizacionException) {
    problemDetails.Title = "Acceso denegado";
    problemDetails.Status = StatusCodes.Status403Forbidden;
    problemDetails.Detail = exception.Message;
} else {
    problemDetails.Title = "Error interno del servidor";
    problemDetails.Status = StatusCodes.Status500InternalServerError;
    problemDetails.Detail = "Ha ocurrido un error inesperado.";
}

// Escribimos la respuesta final
httpContext.Response.StatusCode = problemDetails.Status.Value;
await httpContext.Response.WriteAsJsonAsync(problemDetails, cancellationToken);
return true;
```

Utilizamos el estándar `ProblemDetails` para devolver siempre el mismo formato de `JSON`, evaluando qué tipo de excepción recibimos de la Capa de Aplicación



Modificar Program.cs



En la sección de usings agregar:

```
using EscuelaApi.WebApi;
```

En la sección de configuración de servicios agregar:

```
// Soporte para el formato estándar de errores  
builder.Services.AddProblemDetails();  
  
// Registramos nuestro manejador personalizado  
builder.Services.AddExceptionHandler<ManejadorDeExcepcionesGlobales>();
```

Después de construir app (`var app = builder.Build();`) agregar:

```
// Agregamos el Middleware al principio del Pipeline  
app.UseExceptionHandler();
```




Modificar Program.cs

Ejecutar y probar
en el navegador
<http://localhost:5190/scalar>



Nota Técnica (El Principio Open/Closed):

Nuestra clase `ManejadorDeExcepcionesGlobales` **no es el middleware**.

Cuando llamamos a `app.UseExceptionHandler()`, insertamos el `middleware` nativo de Microsoft (que está `cerrado a la modificación`). Ese `middleware` atrapa el error y le "delega" la traducción a nuestra clase inyectada (`abierto a la extensión`).

Así, Microsoft maneja la infraestructura de red, y nosotros controlamos el formato de la respuesta.

Middleware de excepciones global

POST http://localhost:5190 /api/alumnos

16.3s 400 Bad Request

Error http 400

Detalle del error

Agregando un alumno con email vacío

JSON

```
1 {
2   "nombre": "José",
3   "email": "",
4 }
```

1 {
2 "title": "Error de validación del negocio",
3 "status": 400,
4 "detail": "El formato del email no es válido.",
5 "instance": "/api/alumnos"
6 }

👍

Code Snippet



La Tubería de ASP.NET: Use vs. Map

Al configurar nuestra API, interactuamos con la aplicación (**app**) de dos formas distintas:

1. Los Middlewares (**app.Use...**)

- Son los **filtros o peajes** en la autopista.
- Toda petición pasa por ellos en tránsito.
- *Ejemplos:* `UseExceptionHandler()` (atrapa errores), `UseAuthentication()` (para validar tokens).

2. Los Endpoints (**app.Map...**)

- Son las **salidas o destinos finales**.
- Aquí es donde la petición detiene su viaje hacia adelante, hace un **"giro en U"**, ejecuta nuestro código (Caso de Uso) y la respuesta emprende el camino de regreso.
- *Ejemplos:* `MapGet(...)`, `MapPost(...)`, `MapOpenApi()` (la ruta hacia el manual `JSON`).

Autenticación con JWT

El límite de nuestra seguridad actual

- **El puente temporal:** Hasta ahora, usamos la cabecera `X-Usuario-Id` para saber "quién" hace la petición y verificar sus roles.
- **El problema (Suplantación de Identidad / Spoofing):** Cualquier persona con conocimientos básicos puede abrir `Scalar` o `Postman`, inventar una petición `HTTP` y escribir `X-Usuario-Id: [ID_DEL_ADMINISTRADOR]`.
- Nuestro servidor actualmente lee esa cabecera y le cree ciegamente, permitiéndole eliminar alumnos o cargar notas falsas.
- **La necesidad:** Necesitamos un mecanismo que garantice que el `ID` del usuario viaje de forma **inviolable y verificable** por el servidor.

JSON Web Tokens (JWT) al rescate

- **¿Qué es un JWT?** Es un estándar abierto ([RFC 7519](#)) de la industria que define una forma compacta y autónoma de transmitir información de forma segura entre las partes como un **objeto JSON**.
- **Autónomo:** El **token** contiene toda la información necesaria sobre el usuario (su **ID**, su **rol**). El servidor no necesita ir a consultar la base de datos de usuarios en cada petición para saber quién es.
- **Seguro (Firma Digital):** La información no viaja encriptada (se puede leer), pero viaja **firmada digitalmente**. Si alguien altera una sola letra del **token** en el camino, la firma se rompe y el servidor rechaza la petición.

La Anatomía de un Token JWT

Un **JWT** no es más que una cadena de texto larga dividida en tres partes por un punto (.): **xxxxx.yyyyyy.zzzzz**

1. **Header (Cabecera):** Indica el tipo de token (**JWT**) y el algoritmo de firma usado (ej. **HMAC SHA256**).
2. **Payload (Carga útil):** Contiene los **Claims** (afirmaciones). Aquí guardamos los datos del usuario:
 - **sub** (Subject): El **ID** del usuario.
 - **role**: "Administrador" o "Profesor".
 - **exp** (Expiration): Fecha y hora exacta en la que el token deja de ser válido.
3. **Signature (Firma):** Es el sello de seguridad. Se genera combinando el **Header**, el **Payload** y una **Palabra Secreta** que *solo* conoce nuestro servidor.

La Anatomía de un Token JWT

La analogía del pasaporte

Un **JWT** es como un pasaporte. Los datos dentro están en texto plano (nombre, foto, nacionalidad), cualquiera puede leerlos.

Por eso, **nunca guardamos contraseñas dentro del Payload**, solo información pública de identificación y roles.

Pero lo que lo hace válido es el sello de agua del gobierno (**la firma**). Si cambiamos el nombre, el sello de seguridad delata la falsificación y en **la aduana** (**nuestro servidor**) lo rechazan.

El flujo de Autenticación y Autorización

¿Cómo funciona el ciclo completo en nuestra API?

1. **Login (El nacimiento del token):** El usuario envía un `POST /api/login` con su `Email` y `Contraseña`. El servidor verifica en la base de datos si son correctos. Si lo son, el servidor "fabrica" el `JWT` con su palabra secreta y se lo devuelve al cliente.
2. **Almacenamiento:** El cliente (el navegador web) guarda este `token` (ej. en el `Local Storage`).
3. **Peticiones subsecuentes:** Para crear un examen o eliminar un alumno, el cliente adjunta el token en una `cabecera HTTP` estándar llamada `Authorization`, usando el esquema `"Bearer"` (portador).
 - `Authorization: Bearer <la_secuencia_de_texto_del_jwt>`
4. **Validación:** `ASP.NET Core` intercepta la petición, lee el `token`, verifica que la firma matemática coincida con su palabra secreta y, si es válido, deja pasar la petición hacia nuestro Caso de Uso.

Configurando nuestra Llave Maestra Modificar appsettings.json

Antes de generar tokens, nuestro servidor necesita su **Clave Secreta** y algunos metadatos para la firma (**HMAC SHA256**). Nunca guardamos esto en el código C#, siempre va en la configuración.

En appsettings.json:

```
{
  "ConnectionStrings": {
    "EscuelaDb": "Data Source=Escuela.sqlite"
  },
  "Jwt": {
    "Key": "EstaEsUnaClaveSecretaSuperSeguraParaEscuelaApi2026+",
    "Issuer": "EscuelaApi",
    "Audience": "AlumnosEscuela"
  }
}
```

Key: Debe tener al menos 32 caracteres (256 bits) por seguridad.

Issuer (Emisor): ¿Quién fabricó el token? (Nuestra API).

Audience (Audiencia): ¿Para quién está destinado este token?



Definiendo el Caso de Uso LoginUseCase



El problema del Token y las Capas

Para definir `LoginUseCase` en la capa de **Aplicación**, nos topamos con un problema de dependencias: la capa de **Aplicación** no debe conocer nada acerca de `JWT` que es un detalle de tecnología, no una regla de negocio.

¿La solución elegante? **Inversión de Dependencias**.

Crear la siguiente interfaz en `Aplicacion/Abstracciones/ITokenProvider.cs`:

```
using EscuelaApi.Dominio;
namespace EscuelaApi.Aplicacion;

public interface ITokenProvider
{
    string GenerarToken(Usuario usuario);
}
```

En `Aplicacion/Dtos/Dtos.cs` agregar los siguientes Dtos

```
public record class LoginRequest(string Email, string Password);
public record class LoginResponse(string Token);
```



Definiendo el Caso de Uso LoginUseCase

En Aplicacion/CasosDeUso/LoginUseCase.cs

```
using EscuelaApi.Dominio;
namespace EscuelaApi.Aplicacion;

public class LoginUseCase(IUsuarioRepository repo, ITokenProvider tokenProvider)
{
    public LoginResponse Ejecutar(LoginRequest request)
    {
        DireccionEmail emailIngresado;
        // 1. Delegamos la validación al Dominio intentando crear el Value Object
        try
        {
            emailIngresado = DireccionEmail.Parse(request.Email);
        }
        catch (DominioException) // Atrapamos la regla de dominio rota
        {
            throw new AutorizacionException("El formato del email o la contraseña son incorrectos.");
        }
        // 2. Buscamos al usuario pasándole el Objeto de Valor completo
        var usuario = repo.ObtenerPorEmail(emailIngresado);

        // 3. Validamos credenciales (si falla, lanzamos nuestra excepción ya conocida)
        if (usuario == null || usuario.Password != request.Password)
        {
            throw new AutorizacionException("Email o contraseña incorrectos.");
        }
        // 4. Delegamos la creación del token a la infraestructura
        var token = tokenProvider.GenerarToken(usuario);

        return new LoginResponse(token);
    }
}
```

En EscuelaApi.WebApi Instalar el siguiente paquete Nuget:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

Crear la siguiente clase en WebApi/JwtTokenProvider.cs

```
using Microsoft.IdentityModel.Tokens;  
using System.IdentityModel.Tokens.Jwt;  
using System.Security.Claims;  
using System.Text;  
using EscuelaApi.Aplicacion;  
using EscuelaApi.Dominio;
```

```
namespace EscuelaApi.WebApi;
```

```
public class JwtTokenProvider(IConfiguration config) : ITokenProvider  
{
```

```
    public string GenerarToken(Usuario usuario)  
    {
```

```
        // Guardamos únicamente el ID del usuario para identificarlo
```

```
        var claims = new[] {  
            new Claim("ID", usuario.Id.ToString()),  
        };
```

```
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(config["Jwt:Key"]!));
```

```
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256); // Es similar a un DTO, pero hace  
                                         // algunas validaciones
```

```
        var token = new JwtSecurityToken(  
            issuer: config["Jwt:Issuer"],  
            audience: config["Jwt:Audience"],  
            claims: claims,  
            expires: DateTime.UtcNow.AddHours(2),  
            signingCredentials: creds  
        );
```

```
        return new JwtSecurityTokenHandler().WriteToken(token);
```

```
    }  
}
```

La Implementación de JWT



Registrar los servicios y la ruta para el login en Program.cs



```
// IMPORTANTE: En la sección de configuración registrar el provider
// y el caso de uso
builder.Services.AddScoped<ITokenProvider, JwtTokenProvider>();
builder.Services.AddScoped<LoginUseCase>();

// ... más adelante, los endpoints:

app.MapPost("/api/login", (LoginRequest request, LoginUseCase useCase) =>
{
    // El middleware de excepciones que hicimos atraparé automáticamente
    // la AutorizacionException si la clave está mal y devolverá un
    // HTTP 403.
    var response = useCase.Ejecutar(request);
    return Results.Ok(response);
});
```

Ejecutar y probar
en el navegador
<http://localhost:5190/scalar>

Autenticación con JWT

The screenshot displays a REST client interface with the following components:

- Address Bar:** `http://localhost:5190/scalar/#tag/escuelaapiwebapi/POST/api/login`
- Left Sidebar:** A tree view showing the API structure. The `/api/login` endpoint is selected and highlighted in green. A red arrow points to this endpoint.
- Endpoint Details:**
 - Method:** `POST`
 - Body:** `LoginRequest` (required, application/json)
 - Parameters:**
 - `email` string (required)
 - `password` string (required)
 - Responses:**
 - `200 OK`
- Right Panel:** A dark-themed box containing a `curl` command for testing the endpoint. A red arrow points to the `Test Request` button below the command.

```
POST /api/login
curl http://localhost:5190/api/login \
  --request POST \
  --header 'Content-Type: application/json' \
  --data '{
    "email": "",
    "password": ""
  }'
```
- Response Area:** Below the curl box, the response status `200` is shown, followed by `No Body` and `OK`.

Autenticación con JWT

POST http://localhost:5190 /api/login 306ms 200 OK

Request:

- Headers: accept: */*, content-type: application/json
- Request Body (JSON):

```
1 {
2   "email": "admin@escuela.edu.ar",
3   "password": "admin123"
4 }
```

Response:

- Body (JSON):

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJJCRCi6IjI3MjUyYTMwLTBjOWQ0tN
3 }
```

Annotations:

- Datos para el login**: Points to the request body.
- Nos devuelve un token**: Points to the token in the response body.

Autenticación con JWT

Acceder a la página
www.jwt.io

Encoded Token

☐ Enable auto-focus

> JSON Web Token (JWT)

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJJRRCi6iIjI3MjUyYTMwLTBjO  
WQtNGMyNS1iNGFiLWZlbnNjYyY2I3YSIsImV4cCI6MTc4MTEyNDU1NCwiaXN  
zIjoiaXNjdWVsYUFWaSI6ImF1ZCI6IkpXVCJ9.29JTglRbb  
e8Iq9dxssEkueJ3mDIuPjEdJJ-IF8jXqyw
```

Copiar el token y la
clave secreta para
validarlo

Decoded Header

JSON Claims Breakdown

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Decoded Payload

JSON Claims Breakdown

```
{  
  "ID": "27252a30-0c9d-4c25-b4ab-cf0fe652cb7a",  
  "exp": 1781124154,  
  "iss": "EscuelaApi",  
  "aud": "AlumnosEscuela"  
}
```

JWT Signature Verification (Optional)

Enter the secret used to sign the JWT below:

BASE64URL ENCODED ☐

> Secret

EstaEsUnaClaveSecretaSuperSeguraParaEscuelaApi2026+

✓ Valid secret

✓ Valid JWT

✓ Signature Verified

El token y la firma
son válidos

Últimos pasos

Ahora que la **API** ya sabe **emitir** los pasaportes (el **token**), vamos a enseñarle a **exigirlos y validarlos** en las rutas que queremos proteger.

Estos son los 3 pasos que nos quedan para cerrar el ciclo de seguridad por completo:

- **Paso 1:** Configurar el **Validador de Tokens**
- **Paso 2:** Activar los **Middlewares de Seguridad**
- **Paso 3:** Proteger las **Rutas** y Extraer el **ID**



Paso 1: Configurar el Validador de Tokens en Program.cs

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;
. . .
// En el bloque superior donde registramos los servicios)
// Le decimos a .NET que vamos a usar Autenticación por JWT
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opciones =>
    {
        opciones.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,          // Validar quién lo emitió
            ValidateAudience = true,        // Validar para quién es
            ValidateLifetime = true,         // Validar que no esté vencido
            ValidateIssuerSigningKey = true, // ¡Vital! Validar la firma criptográfica

            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))
        };
    });

// Agregamos el servicio de Autorización (para manejar los roles luego)
builder.Services.AddAuthorization();
```



Paso 2: Activar los Middlewares de Seguridad



En el bloque medio de **Program.cs** (asegurarse de ponerlos DESPUÉS de **app.UseExceptionHandler()** y ANTES de los **app.Map...**):

```
// "Descubre quién es" leyendo el token de la cabecera HTTP  
app.UseAuthentication();
```

```
// "Decide si tiene permiso" para acceder a la ruta solicitada  
app.UseAuthorization();
```



Paso 3: Proteger las Rutas y Extraer el ID

Vamos a tomar uno de los endpoints y le vamos a hacer dos cambios clave:

1. Agregarle `.RequireAuthorization()` al final.
2. Pedirle a .NET que nos inyecte los datos del token usando la clase `ClaimsPrincipal`.

En el bloque de endpoints de Program.cs:

```
using System.Security.Claims; // Necesario para leer el claim "ID"
...
alumnosApi.MapPost("/", (
    AgregarAlumnoRequest request,
    ClaimsPrincipal user, // <- Acá .NET deja los datos del JWT ya validados
    AgregarAlumnoUseCase useCase) =>
{
    // Extraemos el claim "ID" que pusimos adentro del token en el login
    var userIdString = user.FindFirst("ID")?.Value;
    var idUsuario = Guid.Parse(userIdString!);

    var response = useCase.Ejecutar(request, idUsuario);
    return Results.Created($"api/alumnos/{response.Id}", response);
})
.RequireAuthorization(); // <- ¡Esta línea bloquea el acceso a usuarios sin token!
```

Ejecutar y probar en el navegador
<http://localhost:5190/scalar>

Autenticación con JWT

The screenshot shows an API client interface with a POST request to `http://localhost:5190/api/login`. The request body is a JSON object with email and password. The response is a 200 OK status with a JSON body containing a token. Two red boxes with arrows point to the request body and the token in the response.

Request:

```
1 {
2   "email": "admin@escuela.edu.ar",
3   "password": "admin123"
4 }
```

Response:

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIyNzI1MmEzMzAwYzI1LkL
3 }
```

Annotations:

- Autenticarse como administrador en el sistema
- Copiar el token recibido

Autenticación con JWT

POST | http://localhost:5190/api/alumnos | Send | Open API Client

138ms 201 Created

Headers

Key	Value
accept	*/*
content-type	application/json
Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpX...

Request Body

JSON

```
1 {
2   "nombre": "José",
3   "email": "jose@outlook.com"
4 }
```

Response

138ms 201 Created

Body

```
1 {
2   "id": "ef569f8a-f176-4a08-ae8a-797a7ed5d5a8"
3 }
```

Annotations:

- Ingresar los datos del nuevo alumno (points to request body)
- Agregar manualmente:
 - Authorization (en el campo Key)
 - Bearer <token recibido> (en el campo Value)(Es importante que Value comience con "Baerer" seguido de un espacio en blanco, seguido del token)
- El alumno se creó correctamente (points to response body)

Organizando la Documentación con .WithTags() Mejorando la Experiencia del Desarrollador (DX)

En **Minimal APIs**, el método `.WithTags("Nombre")` inyecta metadatos en la especificación **OpenAPI**. No cambia el comportamiento del código, pero agrupa visualmente los endpoints en herramientas como **Scalar** o **Swagger UI**.

¿Por qué usarlo?

- **Orden visual:** Evita tener una lista plana e infinita de rutas mezcladas.
- **Agrupación semántica:** Permite crear "carpetas" o secciones por dominio (Ej: **Alumnos**, **Exámenes**).
- **Testing más fácil:** Ayuda a los equipos de **Frontend** o QA a encontrar rápidamente el **endpoint** que necesitan probar.

Organizando la Documentación con .WithTags() Mejorando la Experiencia del Desarrollador (DX)

Ejemplo de implementación:

```
// 1. Agrupación a nivel de Grupo (Afecta a todas las rutas que cuelguen de aquí)
var alumnosApi = app.MapGroup("/api/alumnos")
    .WithTags("Gestión de Alumnos");
    // Todos los endpoints de alumnos heredarán este tag

// 2. Agrupación a nivel de Endpoint individual (Ideal para endpoints sueltos)
app.MapPost("/api/login", (LoginRequest request, LoginUseCase useCase) =>
{
    // Lógica...
})
.WithTags("Autenticación"); // Crea una sección exclusiva para el Login
```

Organizando la Documentación con .WithTags()

The screenshot shows the Scalar API documentation interface at `http://localhost:5190/scalar/#tag/gestión-de-alumnos`. The left sidebar contains a search bar and a list of tags: Introduction, EscuelaApi.WebApi, Autenticación, **Gestión de Alumnos** (selected), Gestión de Usuarios, Gestión de Exámenes, and Models. The **Gestión de Alumnos** tag is highlighted with a red box, and a red arrow points to it with the text "Gestión de Alumnos".

Below the tag list, the endpoints for the selected tag are displayed:

- `/api/alumnos` GET
- `/api/alumnos` POST
- `/api/alumnos/{alumnoId}/exámenes` GET
- `/api/alumnos/{alumnoId}` DELETE

A red box with the text "Endpoints agrupados en la etiqueta Gestión de Alumnos" points to this list. To the right, the "Operations" section lists the same endpoints with their HTTP methods: GET, POST, GET, and DELETE.

Below the endpoints, the `/api/alumnos` endpoint is selected, showing its responses:

Responses

200 OK

A red arrow points from the "Endpoints agrupados..." box to the `/api/alumnos` endpoint. To the right, a "Shell Curl" section shows the command:

```
curl http://localhost:5190/api/alumnos
```

Below the command, the response is shown as "200" and "No Body".

Antipatrón del "Archivo Monolítico".

En Minimal APIs, como todo se puede hacer directamente en el **Program.cs**, es muy fácil caer en el antipatrón del "**Archivo Monolítico**".

Las **clases estáticas y los métodos de extensión** son la herramienta nativa y más elegante de C# para solucionar esto.

¿La solución? Dividir el archivo usando **Métodos de Extensión** de C#. El objetivo es que **Program.cs** funcione únicamente como un "orquestador de alto nivel", delegando los detalles de implementación a otras clases.

1. Descentralizar las rutas (endpoints) con métodos de extensión para **IEndpointRouteBuilder**
2. Descentralizar el registro de los servicios con métodos de extensión para **IServiceCollection**

Crear distintas clases estáticas en el **proyecto Web API** (en una carpeta que puede llamarse **/Endpoints**) para agrupar las rutas por su contexto de negocio.

En **/Endpoints/AlumnosEndpoints.cs**

```
using System.Security.Claims;
using EscuelaApi.Aplicacion;

namespace EscuelaApi.WebApi;

public static class AlumnosEndpoints
{
    public static void MapAlumnosEndpoints(this IEndpointRouteBuilder app)
    {
        var alumnosApi = app.MapGroup("/api/alumnos").WithTags("Gestión de Alumnos");

        alumnosApi.MapPost("/", (AgregarAlumnoRequest request, ClaimsPrincipal user, AgregarAlumnoUseCase useCase) =>
        {
            . . .
        }).RequireAuthorization(); // <- ¡Esta línea bloquea el acceso a usuarios sin token!

        alumnosApi.MapGet("/", (ConsultarAlumnosUseCase useCase) =>
        {
            . . .
        });
        alumnosApi.MapGet("/{alumnoId:guid}/examenes", (Guid alumnoId, ConsultarExamenesAlumnoUseCase useCase) =>
        {
            . . .
        });

        . . .
    }
}
```

En /Endpoints/ExamenEndpoints.cs

```
using System.Security.Claims;
using EscuelaApi.Aplicacion;

namespace EscuelaApi.WebApi;

public static class ExamenEndpoints
{
    public static void MapExamenEndpoints(this IEndpointRouteBuilder app)
    {
        var examenApi = app.MapGroup("/api/examen").WithTags("Gestión de Exámenes");

        examenApi.MapPost("/", (
            RegistrarExamenRequest request,
            ClaimsPrincipal user,
            RegistrarExamenUseCase useCase) =>
        {
            . . .
        }).RequireAuthorization();

        examenApi.MapPut("/{examenId:guid}", (
            Guid examenId,
            ModificarNotaRequest request, // El Body trae la NuevaNota
            ClaimsPrincipal user,         // Inyectamos la identidad del JWT
            ModificarNotaDeExamenUseCase useCase) =>
        {
            . . .
        }).RequireAuthorization();
    }
}
```

En /Endpoints/LoginEndpoint.cs

```
using System.Security.Claims;
using EscuelaApi.Aplicacion;

namespace EscuelaApi.WebApi;

public static class LoginEndpoint
{
    public static void MapLoginEndpoint(this IEndpointRouteBuilder app)
    {
        app.MapPost("/api/login", (LoginRequest request, LoginUseCase useCase) =>
        {
            . . .
        }).WithTags("Autenticación");
    }
}
```

En /Endpoints/UsuariosEndpoints.cs

```
using System.Security.Claims;
using EscuelaApi.Aplicacion;

namespace EscuelaApi.WebApi;

public static class UsuariosEndpoints
{
    public static void MapUsuariosEndpoints(this IEndpointRouteBuilder app)
    {
        var usuariosApi = app.MapGroup("/api/usuarios").WithTags("Gestión de Usuarios"); ;

        usuariosApi.MapGet("/", (ConsultarUsuariosUseCase useCase) =>
        {
            . . .
        });
    }
}
```

Antipatrón del "Archivo Monolítico".

En `Program.cs`, la sección de mapeos nos quedará:

. . .

```
if (app.Environment.IsDevelopment()) // Solo exponemos esto en modo Desarrollo
{
    app.MapOpenApi(); // Genera el archivo JSON interno
    app.MapScalarApiReference(); // Levanta la interfaz gráfica en /scalar/v1
}
```

```
app.MapGet("/", () => "¡La API de la Escuela está funcionando correctamente!");
```

```
app.MapLoginEndpoint();
app.MapAlumnosEndpoints();
app.MapExámenesEndpoints();
app.MapUsuariosEndpoints();
```

```
app.Run();
```

En la **Capa de Aplicación**, instalar el siguiente paquete Nuget:
`Microsoft.Extensions.DependencyInjection.Abstractions`

Crear la clase estática `Extensiones` para definir el método de extensión `AddAplicacion` que usaremos para registrar en el `Contenedor DI` todos los casos uso de nuestra aplicación.

En el proyecto `EscuelaApi.Aplicacion`:

```
using Microsoft.Extensions.DependencyInjection;

namespace EscuelaApi.Aplicacion;

public static class Extensiones
{
    public static IServiceCollection AddAplicacion(this IServiceCollection servicios)
    {
        servicios.AddScoped<AgregarAlumnoUseCase>();
        servicios.AddScoped<RegistrarExamenUseCase>();
        servicios.AddScoped<ConsultarExamenesAlumnoUseCase>();
        servicios.AddScoped<ModificarNotaDeExamenUseCase>();
        servicios.AddScoped<EliminarAlumnoUseCase>();
        servicios.AddScoped<ConsultarAlumnosUseCase>();
        servicios.AddScoped<ConsultarUsuariosUseCase>();
        servicios.AddScoped<LoginUseCase>();

        return servicios;
    }
}
```


En la **Capa de Infraestructura**, instalar los siguientes paquete Nuget:

`Microsoft.Extensions.DependencyInjection.Abstractions`

`Microsoft.Extensions.Configuration.Abstractions`

Y crear la clase estática `Extensiones`

```
using Microsoft.Extensions.DependencyInjection;
using EscuelaApi.Aplicacion;
using Microsoft.Extensions.Configuration;
using Microsoft.EntityFrameworkCore;

namespace EscuelaApi.Infraestructura;

public static class Extensiones
{
    public static IServiceCollection AddInfraestructura(this IServiceCollection servicios,
                                                       IConfiguration configuration)
    {
        // A. Base de Datos. Extraemos la cadena de conexión del archivo appsettings.json
        var connectionString = configuration.GetConnectionString("EscuelaDb");
        servicios.AddDbContext<EscuelaContext>(opciones =>
            opciones.UseSqlite(connectionString));

        // B. Seguridad (Autorización)
        servicios.AddScoped<IAutorizacionService, AutorizacionService>();

        // C. Patrón Unit of Work y Repositorios
        servicios.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();
        servicios.AddScoped<IAlumnoRepository, AlumnoRepository>();
        servicios.AddScoped<IExamenRepository, ExamenRepository>();
        servicios.AddScoped<IUsuarioRepository, UsuarioRepository>();

        return servicios;
    }
}
```

Antipatrón del "Archivo Monolítico".

Completar los métodos de extensión para que el registro servicios en `Program.cs` pueda escribirse así:

. . .

```
var builder = WebApplication.CreateBuilder(args);
```

```
// ***** Registro de Servicios en el Contenedor DI *****
```

```
builder.Services.AddOpenApi()  
    .AddAplicacion()  
    .AddInfraestructura(builder.Configuration)  
    .AddAutorizacionJWT(builder.Configuration)  
    .AddManejoDeExcepciones();
```

```
var app = builder.Build();
```

Fin teoría 12

Práctica sobre la teoría 12

Esta teoría no tiene práctica asociada. Los conceptos vistos aquí serán utilizados en el Trabajo Práctico Final